

Unit – 1

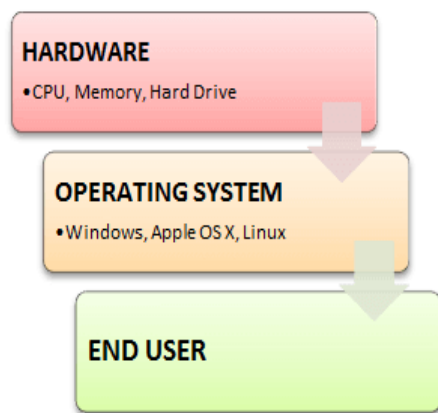
Chapter 1 - Introduction

Syllabus:

Introduction: Definition, Computer System Components, User View, System View and System Goals, Batch Systems, Multi Programmed Systems, Time-Sharing Systems, Real-Time Systems, System Components, Operating System Services.

Definition:

An Operating System (OS) is software that acts as an interface between the user and the computer hardware. It manages computer resources like CPU, memory, storage, and input/output devices, while providing an environment for application software to run.



Examples: Windows, Linux, and Android.

Computer System Components

A computer system is made up of several key components that work together to perform various tasks. These components can be categorized into hardware, software, and users.

Hardware:

- Central Processing Unit (CPU): The primary component that executes instructions and processes data. It consists of the Arithmetic Logic Unit (ALU) and Control Unit (CU).
- Memory: Temporary storage (RAM) that holds data and instructions the CPU needs while performing operations.
- Storage Devices: These provide long-term storage of data, such as Hard Drives (HDD), Solid-State Drives (SSD), optical discs (CD/DVD), etc.
- Input Devices: Devices through which data is entered into the system (e.g., keyboard, mouse, scanner).
- Output Devices: Devices that present data from the system to the user (e.g., monitor, printer, speakers).
- Motherboard: The primary circuit board that connects all the hardware components together.
- Peripheral Devices: Additional devices like printers, external drives, USBs, etc., connected to the computer.

Software:

The programs and applications that run on the hardware, enabling the user to perform specific tasks.

- **System Software:** Software that manages hardware and provides a platform for application software. Examples: Operating systems, device drivers.
- **Application Software:** Software designed for specific tasks to assist users. Examples: Microsoft Word, web browsers

Users: Individuals or systems that interact with the computer, either directly (via a GUI) or indirectly (via automated systems or applications).

User View and System View of Operating System



1)User View of Operating System:

The **user view** represents how the operating system interacts with the user. It focuses on making the system easy to use, providing an interface (like GUI or CLI) for running applications, managing files, and performing tasks. Users are unaware of the internal operations and see only the tools and services needed for their work.

- Execution of a program
- Access to I/O devices
- Controlled access to files
- Error detection (Hardware failures, and software errors)

Example: A user opens a browser, edits a document, or copies files using the OS interface without understanding how memory or process management works.

2)System View of Operating System:

The **system view** focuses on the internal role of the operating system as a resource manager. It deals with the allocation and coordination of hardware resources like CPU, memory, and I/O devices. The OS ensures processes are executed efficiently and securely, optimizing the system's performance.

- Hardware upgrades
- New services
- Fixes the issues of resources
- Controls the user and hardware operations

Example: The OS schedules processes, manages memory, and coordinates hardware access without user intervention.

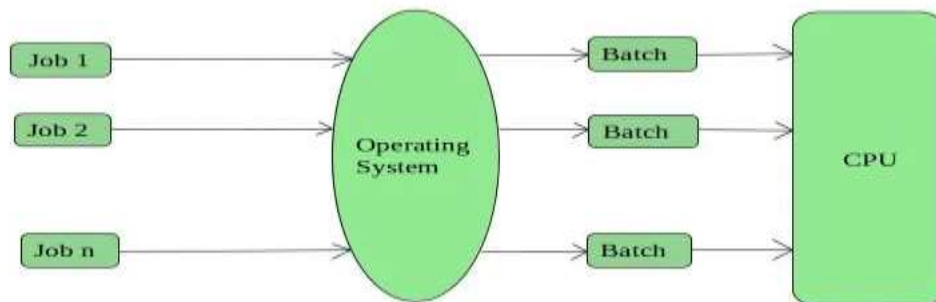
Operating System Goals:

- **Efficiency:** The OS should make sure the computer's resources (like CPU, memory, and storage) are used effectively, so nothing is wasted.

- **Reliability:** The OS should ensure the system works properly and doesn't crash or fail unexpectedly.
- **Security:** The OS needs to protect the system from unauthorized access and keep data safe from being tampered with.
- **Consistency:** The OS should provide a stable and predictable experience for users, no matter what changes are made in the background.
- **Performance:** The OS should make sure tasks are completed as quickly as possible and the system responds fast to user commands.
- **Scalability:** The OS should be able to handle more users or tasks as the system grows, without slowing down.

Batch operating system

This type of operating system does not interact with the computer directly. There is an operator which takes similar jobs having the same requirements and groups them into batches. It is the responsibility of the operator to sort jobs with similar needs. Batch Operating System is designed to manage and execute a large number of jobs efficiently by processing them in groups.



Advantages

- Multiple users can share the batch systems.
- The idle time for the batch system is very less.
- It is easy to manage large work repeatedly in batch systems.

Disadvantages

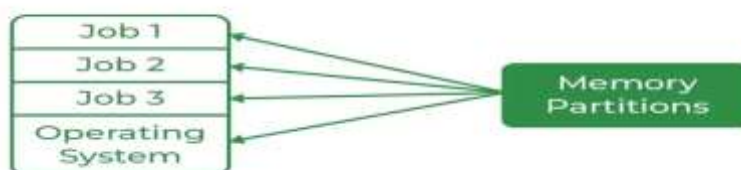
- Batch systems are hard to debug.
- It is sometimes costly.

Multi programming operating systems

Multiprogramming is an extension to batch processing where the CPU is always kept busy. Each process needs two types of system time: CPU time and IO time.

In a multiprogramming environment, when a process does its I/O, The CPU can start the execution of other processes. Therefore, multiprogramming improves the efficiency of the system.

Multiprogramming



Advantages

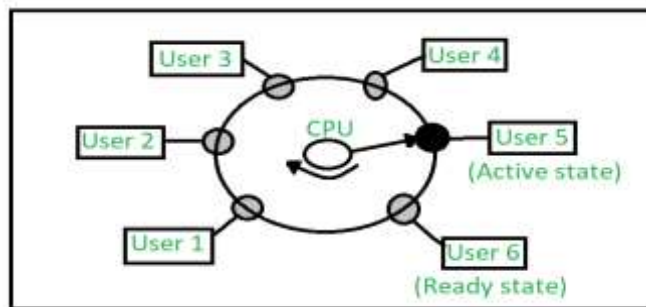
- Multi Programming increases the Throughput of the System.
- It helps in reducing the response time.

Disadvantages

- There is not any facility for user interaction of system resources with the system.

Time-Sharing Operating Systems

Each task is given some time to execute so that all the tasks work smoothly. Each user gets the time of the CPU as they use a single system. These systems are also known as Multitasking Systems. The task can be from a single user or different users also. The time that each task gets to execute is called quantum. After this time interval is over OS switches over to the next task.



Advantages

- Each task gets an equal opportunity.
- Fewer chances of duplication of software.
- CPU idle time can be reduced.

Disadvantages

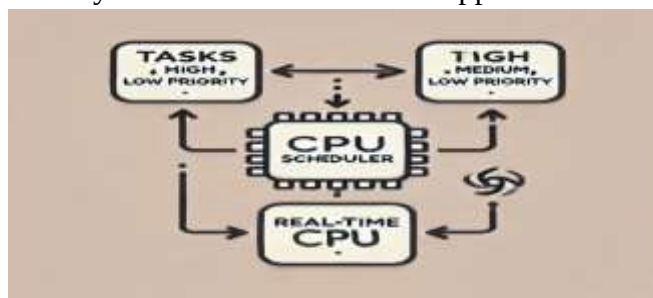
- Reliability problem.
- One must have to take care of the security and integrity of user programs and data.
- Data communication problem.

Real-Time Operating Systems (RTOS)

A Real-Time Operating System is an OS designed to process data and respond to inputs immediately within a fixed time constraint. It is used in systems where timing is critical, and even a small delay can lead to failure or performance issues.

Types of Real-Time Operating Systems

- Hard Real-Time Systems: Hard Real-Time OSs are meant for applications where time constraints are very strict and even the shortest possible delay is not acceptable.
- Soft Real-Time Systems: These OSs are for applications where time-constraint is less strict.



Advantages:

1. **Fast Response:** Tasks are executed quickly and on time.
2. **Task Prioritization:** Critical tasks are given priority and executed first.
3. **Reliability:** Provides consistent and predictable performance.
4. **Efficient Resource Use:** Manages CPU, memory, and other resources effectively.
5. **Low Latency:** Reduces delays, ensuring immediate task execution.

Disadvantages:

1. **Expensive:** RTOS can be costly to develop and maintain.
2. **Complex Design:** Requires specialized programming and hardware.
3. **Limited Tasks:** Strict time constraints can limit multitasking.
4. **Resource Intensive:** Needs more CPU power and memory for real-time processing.
5. **Difficult to Debug:** Finding and fixing errors in real-time systems is challenging.

System Components

various components work together to manage hardware and software resources while providing a user-friendly environment to perform tasks



Process management handles the execution of processes, including their creation, scheduling, and termination. It ensures efficient multitasking by allocating CPU time and other resources to active processes. For example, it allows you to browse the web while playing music in the background.

File management is responsible for organizing, storing, and retrieving data in files and directories. It allows users to save documents, access files, and manage data securely, ensuring efficient usage of storage devices.

Network management facilitates communication between systems over a network. It handles data transfer, sharing resources, and maintaining secure connections, enabling activities such as accessing the internet or sharing files between devices.

Memory management focuses on allocating and managing the system's RAM for running processes and applications. It ensures optimal memory usage and avoids conflicts, such as loading programs into memory and freeing space once they are closed.

Secondary storage management deals with long-term data storage on devices like hard drives and SSDs. It manages how data is stored and retrieved, ensuring storage space is efficiently utilized.

I/O device management oversees the interaction between the system and input/output devices, such as keyboards, printers, and monitors. It ensures smooth communication and coordination between hardware devices and the software.

Security management protects the system against unauthorized access and threats like viruses or malware. It enforces access control mechanisms, encryption, and authentication protocols to safeguard user data and system resources.

The **command interpreter system** provides an interface (CLI or GUI) for users to interact with the OS. It interprets user commands and executes them, enabling tasks like opening files, running applications, and configuring settings.

All these components work together in the OS to provide a reliable, secure, and efficient environment for both system operations and user activities.

Operating System Services

1. Program Execution:

- The OS provides an environment for running programs.
- It loads the program into memory, schedules it for execution, allocates necessary resources, and ensures smooth operation.
- If a program terminates abnormally (e.g., due to an error), the OS handles clean-up tasks like releasing memory and notifying the user.

2. I/O Operations:

- Input and output operations are essential for interacting with peripherals (e.g., printers, keyboards, disks).
- The OS abstracts hardware complexities and provides interfaces for applications to perform I/O operations.
- It ensures that multiple programs can share devices without conflicts.

3. File System Access:

- The OS organizes and manages files on storage devices.
- It provides services for creating, deleting, reading, writing, and organizing files.
- Additionally, it manages directories, access permissions, and handles errors like disk full or file corruption.

4. Error Detection and Handling:

- The OS constantly monitors the system for hardware failures, illegal memory access, or invalid operations.
- It logs errors, attempts recovery, and informs the user or administrator.
- For example, if a printer is not responding, the OS might queue tasks or retry the operation.

5. Communication:

- Processes need to communicate for sharing data or coordinating actions.
- The OS provides interprocess communication (IPC) mechanisms like message passing, shared memory, and semaphores.
- In networked systems, the OS facilitates communication over protocols like TCP/IP.

6. Resource Allocation:

- The OS acts as a resource manager for CPU time, memory, I/O devices, and storage.
- It uses algorithms like scheduling (e.g., Round-Robin, Priority Scheduling) to ensure fair and efficient resource distribution among processes.
- This avoids deadlock, starvation, or overuse of resources.

7. Security and Protection:

- The OS prevents unauthorized access to system resources and data.
- It enforces user authentication (e.g., passwords, biometrics) and permissions for accessing files or devices.
- It also isolates processes from interfering with each other's operations.

8. System Monitoring and Accounting:

- The OS keeps track of resource usage by users and processes (e.g., CPU time, memory).
- This information can be used for system performance analysis, optimization, or billing in multi-user environments.
- It also logs system activities for troubleshooting or auditing.

These services ensure that the system operates efficiently, reliably, and securely while providing a user-friendly interface for program execution

Unit – 1

Chapter 2 - Process

Syllabus:

Process: Process Concept, Process State Diagram Process Control Block, Process Scheduling- Scheduling Queues, Scheduler, Cooperating Process, Interprocess Communication.

PROCESS

“A process is basically a program in execution”. The execution of a process must progress in a sequential fashion.

A process defines the fundamental unit of computation for the computer Components:

Object: Program code to be executed.

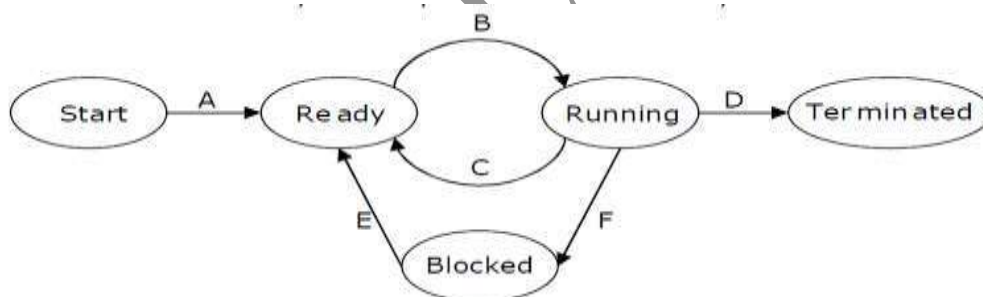
Data: The data is be used in execution of a program.

Resources: While executing the program the resource it may required.

Status of the process execution: Verify the status of program execution.

PROCESS STATE

Process state is defined as the current activity of the process. When process executes, it changes state. In general, a process can have one of the following five states at a time.



1 Start

This is the initial state when a process is first started/created.

2 Ready

The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after Start state or while running it by but interrupted by the scheduler to assign CPU to some other process.

3 Running

Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.

4 Waiting / Blocked

Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.

5 Terminated or Exit

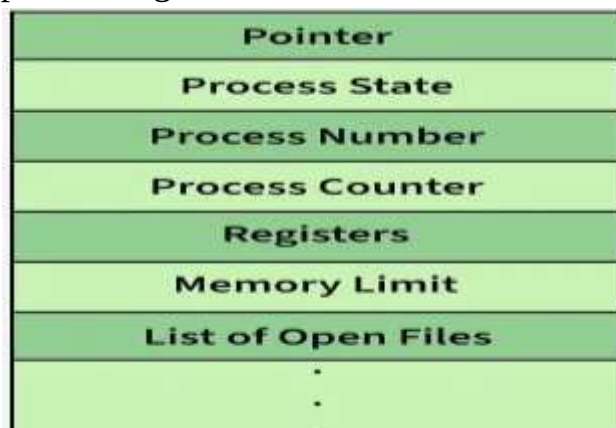
Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.

PROCESS CONTROL BLOCK (PCB)

“A Process Control Block is a data structure maintained by the Operating System for every process”. The PCB is identified by an integer process ID (PID).

The process control keeps track of many important pieces of information needed to manage processes efficiently.

Here is a simplified diagram of a PCB –



- **Pointer:** It is a stack pointer that is required to be saved when the process is switched from one state to another to retain the current position of the process.
- **Process state:** It stores the respective state of the process.
- **Process number:** Every process is assigned a unique id known as process ID or PID which stores the process identifier.
- **Program counter:** Program Counter stores the counter, which contains the address of the next instruction that is to be executed for the process.
- **Register:** Registers in the PCB, it is a data structure. When a process is running and its time slice expires, the current value of process specific registers would be stored in the PCB and the process would be swapped out. When the process is scheduled to be run, the register values is read from the PCB and written to the CPU registers. This is the main purpose of the registers in the PCB.
- **Memory limits:** This field contains the information about memory management system used by the operating system. This may include page tables, segment tables, etc.
- **List of Open files:** This information includes the list of files opened for a process.

PROCESS SCHEDULING

“The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process”.

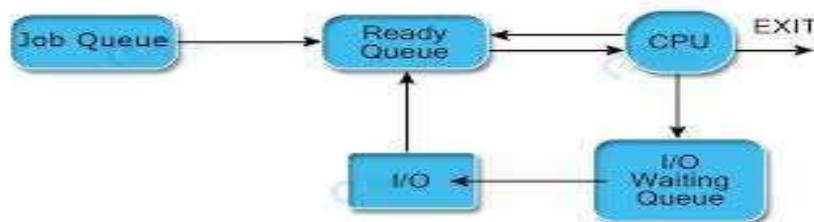
Process Scheduling Queues

The OS maintains all PCBs in Process Scheduling Queues.

The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue.

When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.

The Operating System maintains the following important process scheduling queues



1. Job Queue: This is where processes wait to be admitted into the system. It contains all processes that are waiting to be loaded into memory for execution.
2. Ready Queue: Once a process is loaded into memory and is ready to run, it is moved to the ready queue. The processes in this queue are waiting for CPU time to execute. The CPU scheduler selects one of them for execution based on scheduling algorithms.
3. CPU: The Central Processing Unit (CPU) performs the actual execution of the processes. Once a process is selected from the ready queue, it starts executing in the CPU. After execution, the process can either terminate or move to another queue.
4. I/O Waiting Queue: Processes that need to perform Input/Output operations (like reading from a disk or waiting for a user input) are moved to the I/O waiting queue. These processes are waiting for I/O operations to complete before they can continue execution.
5. I/O: This represents the actual Input/Output operation that a process is performing. While a process is executing an I/O operation, it cannot be executed by the CPU.
6. Exit: Once a process completes its execution, it moves to the exit state, signifying that it has finished its lifecycle.

The process flow involves a job entering the job queue, moving through the ready queue for execution by the CPU, possibly being sent to the I/O waiting queue if it needs to perform an I/O operation, and eventually exiting after completion.

SCHEDULERS

“Their main task is to select the jobs to be submitted into the system and to decide which process to run”. Schedulers are of three types –

1. Long-Term Scheduler
2. Short-Term Scheduler
3. Medium-Term Scheduler

Long-Term Scheduler (Job Scheduler)

The long-term scheduler decides which processes get into the system to be executed. It picks processes from the job queue and loads them into memory for execution.

Long-Term Scheduler controls which processes enter the system and how many are in memory.

- It controls how many processes are in memory at any time, which is known as degree of multiprogramming.
- It ensures a balance between processes that use a lot of CPU time (CPU-bound) and those that spend more time waiting for I/O operations (I/O-bound).
- It's slower than the short-term scheduler because it runs less often.
- In time-sharing systems, this scheduler might be minimal or absent.
- When a process moves from new to ready, the long-term scheduler is involved.

Short-Term Scheduler (CPU Scheduler)

The short-term scheduler decides which process from the ready queue gets to use the CPU next. It's responsible for allocating CPU time to processes that are ready to execute.

Short-Term Scheduler selects which process should run next and allocates CPU time.

- It selects processes that are in the ready state and moves them to the running state.
- The short-term scheduler is fast because it needs to make decisions quickly to keep the CPU busy.
- It helps maximize the overall system performance by efficiently managing CPU resources.
- It makes quick decisions on which process should run next, based on scheduling algorithms (e.g., Round Robin, First Come First Serve).

Medium-Term Scheduler

The medium-term scheduler is responsible for managing processes that are swapped out of memory temporarily. When a process cannot make progress (e.g., waiting for I/O), it is moved out of memory to secondary storage to free up space for other processes.

Medium-Term Scheduler handles processes that are swapped out of memory temporarily to balance system load.

- It helps in swapping processes: moving processes between main memory and secondary storage (like hard disk).
- This process is useful when the system is overloaded, and there isn't enough memory to keep all processes active.
- The medium-term scheduler reduces the degree of multiprogramming by swapping out processes.
- When the swapped-out process is ready again, it is swapped back into memory so it can continue execution.

Comparison Between the Schedulers:

Feature	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
Main Role	Decides which processes enter the system.	Decides which process gets CPU time.	Manages processes swapped in and out of memory.
Speed	Slower (it runs less often).	Fastest (quick decisionmaking).	In between, as it handles swapping of processes.
Control over Processes	Controls how many processes are in memory.	Provides less control over how many processes run.	Reduces the number of processes in memory.
Use in Time-Sharing Systems	Rare or minimal.	Minimal (allocates CPU time to many processes).	Part of time-sharing systems for efficient memory use.
Task	Loads processes from the job queue into memory.	Allocates CPU time to ready processes.	Swaps processes in/out of memory based on system load.

CO-OPERATING PROCESSES

Co-operating processes are processes that work together to achieve a common task. These processes share data or resources, and their execution may depend on one another. A co-operating process is one that can affect or be affected by other processes during execution. If Process 1 is reading data from a file while Process 2 is writing to the same file, their behavior affects one another, making them co-operating processes.

Benefits of Co-operating Processes:

Sharing of Information:

- Co-operating processes can share data such as files, memory, or variables.
- Example: A database management system where one process reads data, and another writes it.
- The system must allow safe concurrent access to shared resources to avoid conflicts.

Increased Computation Speed:

- When multiple processors or cores are available, co-operating processes can execute tasks concurrently.
- Example: A multi-threaded program where different threads perform different parts of a computation in parallel.

Modularity:

- Complex systems can be divided into smaller, manageable modules, each handled by a separate process.
- Example: A compiler can have separate processes for lexical analysis, syntax analysis, and code generation.

Convenience:

- Co-operating processes simplify solving complex problems by dividing tasks among multiple processes.
- Example: A client-server model where the server handles requests from multiple clients.

Two processes writing to a terminal:

- Process 1: `printf("abc")`
- Process 2: `printf("CBA")`

Possible outputs:

- CBAabc
- abCBA
- abCeBA

INTERPROCESS COMMUNICATION (IPC)

Interprocess Communication (IPC) is a mechanism that allows processes to exchange data and information.

Processes in an operating system may either be independent or cooperating.

- **Independent Process:**
A process that cannot affect or be affected by other processes in the system.
It does not share data with any other process.

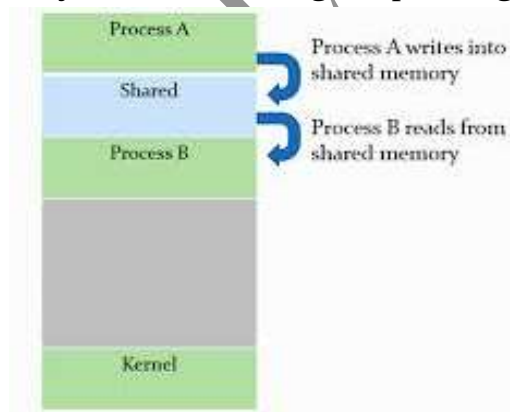
- **Cooperating Process:**
A process that can affect or be affected by other processes.
It shares data or resources with other processes.

Reasons for Process Cooperation:

1. **Information Sharing:**
 - Multiple users or processes may need to access shared information, such as a file.
 - The system should allow concurrent access to such shared resources.
2. **Computation Speedup:**
 - Large tasks can be divided into smaller subtasks, which can be executed in parallel. ○ Speedup is achieved only if the system has multiple processing elements, such as CPUs or I/O channels.
3. **Modularity:**
 - Systems can be designed in a modular way, dividing functionality into separate processes or threads.
4. **Convenience:**
 - A single user may work on multiple tasks at the same time, such as editing, printing, and compiling, all running in parallel.

Models of Interprocess Communication (IPC):

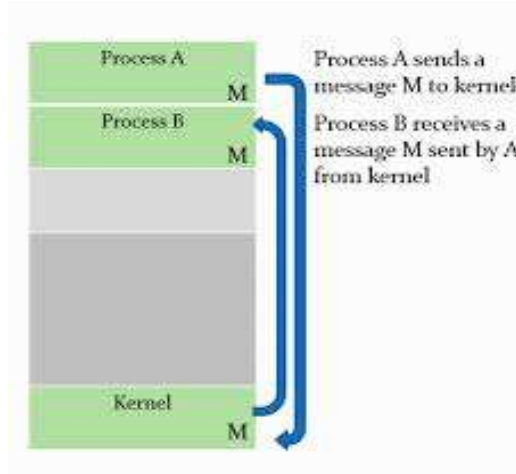
1. **Shared Memory Model:**
A region of memory is shared among cooperating processes.



- **How It Works:**
 - ✦ Processes exchange information by reading and writing data in the shared memory region.
- **Advantages:**
 - ✦ High speed and efficient communication.
 - ✦ Faster than message passing as communication happens at memory speeds without kernel intervention.
- **Disadvantages:**
 - ✦ Requires proper synchronization mechanisms (e.g., semaphores, mutexes) to avoid conflicts.

2. Message Passing Model:

Processes communicate by sending and receiving messages.



○ How It Works:

- ✦ Data is exchanged explicitly through messages between processes.

○ Advantages:

- ✦ Easier to implement, especially for communication between processes on different computers.
- ✦ Useful for exchanging small amounts of data since no shared memory conflicts arise.
 - Disadvantages:
- ✦ Slower than shared memory because it typically requires system calls and kernel intervention.

Comparison of Shared Memory and Message Passing:

Feature	Shared Memory	Message Passing
Speed	Faster (no kernel intervention during access).	Slower (requires kernel intervention).
Complexity	Requires synchronization to avoid conflicts.	Simpler as no shared memory conflicts.
Use Case	Suitable for large amounts of data.	Suitable for smaller amounts of data.
Implementation	Harder to implement across different systems.	Easier to implement in distributed systems.

Unit – 1

Chapter 3 - CPU Scheduling

Syllabus:

CPU Scheduling: Basic Concepts, Preemptive and Non-Preemptive Scheduling, Scheduling Criteria, Scheduling Algorithms-FCFS, Shortest Job First Priority Scheduling, Round Robin Scheduling.

BASIC CONCEPT:

- CPU scheduling is a process which allows one process to use the CPU while the execution of another process is on hold due to unavailability of any resource like I/O.
- “Select the process from ready queue and CPU allocate that process”.
- The aim of CPU scheduling is to make the system efficient and fast.

PREEMPTIVE SCHEDULING

- In the preemptive scheduling, a process can be interrupted when it is being executed.
- Preemptive scheduling works by dividing the time slots of CPU to a given process
- **Examples:** Shortest Remaining Time First (SRTF), Round Robin (RR).
- Suitable for time-sharing systems and ensures better response time.

NON-PREEMPTIVE SCHEDULING

- In the non-preemptive scheduling, a process cannot be interrupted until it terminates itself or its time is over.
- Non-preemptive scheduling takes the CPU and holds it till the process gets terminated or is pushed to the waiting state.
- **Examples:** First-Come, First-Served (FCFS), Shortest Job Next (SJN).
- Simpler but may cause longer waiting times.

SCHEDULING CRITERIA:

Many algorithms exist for CPU scheduling. Various criteria have been suggested for comparing these CPU scheduling algorithms. Common criteria include:

1. **CPU utilization:** This may range from 0% to 100% ideally. In real systems it ranges from 40% for lightly loaded systems to 90% for heavily-loaded systems.
2. **Throughput:** The number of processes completed per time unit is throughput. Long processes may be of the order of one process per hour whereas in case of short processes, throughput may be 10 or 12 processes per second.

3. **Turnaround time:** The interval of time between submission and completion of a process is called turnaround time. It includes execution time and waiting time.

$$\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$$

4. **Waiting time:** Sum of all the times spent by a process at different instances waiting in the ready queue is called waiting time.

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

5. **Response time:** In an interactive process the user is using some output generated while the process continues to generate new results.

$$\text{Response Time} = \text{First Response Time} - \text{Arrival Time}$$

SCHEDULING ALGORITHMS

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms.

- First-Come First-Served (FCFS) Scheduling
- Shortest-Job-First (SJF) Scheduling
- Priority Scheduling
- Round Robin (RR) Scheduling

Turn Around Time (T.A.T) = (Completion Time) – (Arrival Time)

Waiting Time (W.T) = (Turn Around Time) – (Burst Time)

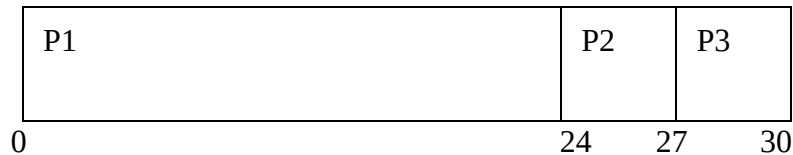
Response Time (R.T) = (First Arrival Time) – (Arrival Time)

FIRST COME FIRST SERVE (FCFS)

- Its implementation is based on FIFO queue.
 - Definition: FCFS is an operating system scheduling algorithm that automatically executes queued requests and processes by order of their arrival
 - It supports non-preemptive scheduling algorithm.
 - It is the simplest form of a CPU scheduling algorithm
 - It is a Non-Preemptive CPU scheduling algorithm, so after the process has been allocated to the CPU, it will never release the CPU until it finishes executing.
 - Poor in performance as average wait time is high
- 1) Consider a set of three processes P1, P2 and P3 arriving at time instant 0 and having CPU burst times as shown below calculate Average waiting time and average turnaround time?

Process	Burst time (msecs)
P1	24
P2	3
P3	3

The Gantt chart below shows the result.



The waiting time for process P1 = 0 msecs

P2 = 24 msecs

P3 = 27 msecs

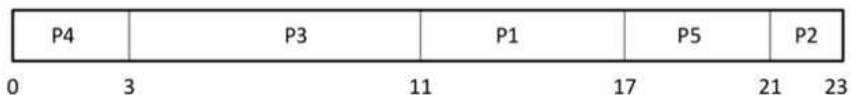
Average waiting time = $(0 + 24 + 27) / 3 = 51 / 3 = 17$ msecs.

P1 completes at the end of 24 msecs, P2 at the end of 27 msecs and P3 at the end of 30 msecs.

Average turnaround time = $(24 + 27 + 30) / 3 = 81 / 3 = 27$ msecs.

- 2) Consider a set of three processes P1, P2, P3, P4 and P5 having arriving at time and CPU burst times as shown below calculate Average waiting time?

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4



$$\frac{0+2+9+13+16}{5}$$

Average Waiting Time = $\frac{40}{5} = 8$.

- Thus if processes with smaller CPU burst times arrive earlier, then average waiting and average turnaround times are less.
- In the meanwhile, I/O bound processes finish I/O and wait for CPU thus leaving the I/O devices idle.
- The CPU-bound process releases the CPU as it goes for an I/O.

- I/O bound processes have short CPU bursts and they execute and go for I/O quickly. The CPU is idle till the CPU-bound process finishes the I/O and gets hold of the CPU.
- The above cycle repeats. This is called the convoy effect. Here small processes wait for one big process to release the CPU.
- Since the algorithm is non-preemptive in nature, it is not suited for timesharing systems.

SHORTEST JOB FIRST (SJF)

- This is also known as **shortest job first**, or SJF
- This is a non-preemptive, pre-emptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processor should know in advance how much time process will take.

1) As an example, consider the following set of processes P1, P2, P3, P4 and their CPU burst times:

Process	Burst time (msecs)
P1	6
P2	8
P3	7
P4	3

Using SJF algorithm, the processes would be scheduled as shown below.

P4	P1	P3	P2	
0	3	9	16	24

Average waiting time = $(0 + 3 + 9 + 16) / 4 = 28 / 4 = 7$ msecs.

Average turnaround time = $(3 + 9 + 16 + 24) / 4 = 52 / 4 = 13$ msecs.

- For a given set of processes, the algorithm gives the minimum average waiting and turnaround times. This is because, shorter processes are scheduled earlier than longer ones and hence waiting time for shorter processes decreases more than it increases the waiting time of long processes.
- The main disadvantage with the SJF algorithm lies in knowing the length of the next CPU burst. In case of long-term or job scheduling in a batch system, the time required to complete a job as given by the user can be used to schedule.
- SJF algorithm is therefore applicable in long-term scheduling.

- SJF algorithm could be either preemptive or non-preemptive. If a new process joins the ready queue with a shorter next CPU burst than what is remaining of the current executing process, then the CPU is allocated to the new process.
- In case of non-preemptive scheduling, the current executing process is not preempted and the new process gets the next chance, it being the process with the shortest next CPU burst.

Given below are the arrival and burst times of four processes P1, P2, P3 and P4.

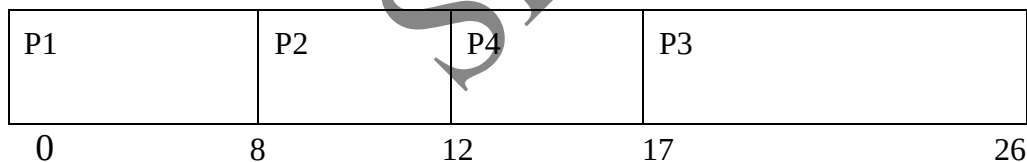
Process	Arrival time (msecs)	Burst time(msecs)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

1. If SJF **preemptive** scheduling is used, the following Gantt chart shows the result.



Average waiting time = $((10 - 1) + 0 + (17 - 2) + (5 - 3)) / 4 = 26 / 4 = 6.5$ msecs.

2. If **non-preemptive** SJF scheduling is used, the result is as follows:



Average waiting time = $((0 + (8 - 1) + (12 - 3) + (17 - 2)) / 4 = 31 / 4 = 7.75$ msecs.

PRIORITY BASED SCHEDULING

- Priority-based algorithms can be either preemptive or non-preemptive the most common scheduling algorithms in batch systems.
- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

- 1) In the following example, we will assume **lower numbers to represent higher priority**.

Process	Priority	Burst time (msecs)
P1	3	10
P2	1	1
P3	3	2
P4	4	1
P5	2	5

Using priority scheduling, the processes are scheduled as shown below:

P2	P5	P1	P3	P4	
0	1	6	16	18	19

Average waiting time = $(6 + 0 + 16 + 18 + 1) / 5 = 41 / 5 = 8.2$ msecs.

- In case of preemptive scheduling, if a new process joins the ready queue with a priority higher than the process that is executing, then the current process is preempted and CPU allocated to the new process. But in case of nonpreemptive algorithm, the new process having highest priority from among the ready processes is allocated the CPU only after the current process gives up the CPU.

2) Consider the following table of arrival time, Priority, and burst time for five processes **P1, P2, P3, P4,** and **P5. (higher numbers to represent higher priority)**

Process	Arrival Time	Priority	Burst Time
P1	0 ms	3	3 ms
P2	1 ms	2	4 ms
P3	2 ms	4	6 ms
P4	3 ms	6	4 ms
P5	5 ms	10	2 ms

P1	P2	P2	P2	P1	P3	P4	P5	
0	1	2	3	5	7	13	17	19

Here,

- **Total Turn Around Time** = $7 + 4 + 11 + 14 + 14 = 50$ ms
- **Average Turn Around Time** = $(\text{Total Turn Around Time})/(\text{no. of processes}) = 50/5 = 10.00$ ms
- **Total Waiting Time** = $4 + 0 + 5 + 10 + 12 = 31$ ms
- **Average Waiting Time** = $(\text{Total Waiting Time})/(\text{no. of processes}) = 31/5 = 6.20$ ms
- **Total Response Time** = $0 + 0 + 5 + 10 + 12 = 27$ ms
- **Average Response Time** = $(\text{Total Response Time})/(\text{no. of processes}) = 27/5 = 5.40$ ms

ROUND ROBIN SCHEDULING

- Round Robin is the preemptive process scheduling algorithm.
- Each process is provided a fix time to execute, it is called a **quantum**.
- Once a process is executed for a given time period, it is preempted and other process executes for a given time period.
- Context switching is used to save states of preempted processes.

1) Consider the same example explained under FCFS algorithm.

Process	Burst time (msecs)
P1	24
P2	3
P3	3

Let the duration of a time slice be 4 msecs, which is to say CPU switches between processes every 4 msecs in a round-robin fashion. The Gantt chart below shows the scheduling of processes.

P1	P2	P3	P1	P1	P1	P1	P1	
0	4	7	10	14	18	22	26	30

Average waiting time = $(4 + 7 + (10 - 4)) / 3 = 17 / 3 = 5.66$ msecs.

UNIT-1

Reference questions

2 Marks Questions

1. Define Operating System.
 2. What are the main components of a computer system?
 3. What is the difference between user view and system view of OS?
 4. Define Batch Processing System.
 5. What is Multiprogramming?
 6. Mention any two features of Time-Sharing System.
 7. What is Real-Time Operating System?
 8. What is a process in OS?
 9. Define Process Control Block (PCB).
 10. What is a Scheduling Queue?
 11. Define Preemptive Scheduling.
 12. Name two CPU scheduling algorithms.
 13. What is Turnaround Time?
 14. Define Interprocess Communication (IPC).
 15. What is Round Robin Scheduling?
-

4 Marks Questions

1. Explain the components of a computer system.
 2. Compare Batch System and Time-Sharing System.
 3. Explain Process States with a neat diagram.
 4. What are the differences between Preemptive and Non-preemptive Scheduling?
 5. List and explain the different types of schedulers.
 6. Describe CPU scheduling criteria.
 7. Write short notes on Interprocess Communication.
 8. What are the different types of system goals?
-

5 Marks Questions:

1. Explain the functions of an Operating System.
2. With a neat diagram, explain the Process State Diagram.
3. Describe different types of CPU Scheduling Algorithms briefly.
4. Explain Priority Scheduling with an example.

5. Discuss the structure and components of Process Control Block.
 6. Compare FCFS and Round Robin Scheduling with examples.
 7. Explain Time Sharing and Real-Time Systems with examples.
-

8 Marks Questions

1. Explain various types of Operating Systems in detail.
2. With a neat diagram, explain the Process Life Cycle and PCB structure.
3. Describe different CPU Scheduling Algorithms with examples and Gantt charts (FCFS, SJF, Priority, RR).
4. Discuss Process Scheduling in detail.
5. What is Interprocess Communication? Explain message passing and shared memory mechanisms with diagrams.
6. Explain the role of schedulers in CPU scheduling and how scheduling queues are managed.
7. Compare and contrast different Scheduling Algorithms based on Scheduling Criteria.